

Listy w C#

List<T> – od podstaw do praktyki

Plan lekcji

- | | | |
|----|------------------------------------|--------|
| 01 | Dlaczego listy? Tablica vs List<T> | 8 min |
| 02 | Deklaracja i inicjalizacja | 10 min |
| 03 | Metody: Add, Remove, Sort... | 12 min |
| 04 | Wyszukiwanie i LINQ | 8 min |
| 05 | Ćwiczenia praktyczne | 7 min |

Problem z tablicami

Tablica – problem

```
// Rozmiar ustalony z góry  
int[] oceny = new int[5];  
  
// Chcemy dodać 6. element?  
// Trzeba nową tablicę + kopiowanie!  
int[] nowe = new int[6];  
Array.Copy(oceny, nowe, 5);
```

✗ Ograniczone, wymaga ręcznej pracy

List<T> – rozwiązanie

```
// Dynamiczny rozmiar!  
var oceny = new List<int>();  
  
oceny.Add(5);  
oceny.Add(4);  
oceny.Add(3);  
// Ile chcemy – bez limitu!  
oceny.Add(6); // OK!
```

✓ Dynamiczna, wygodna, elastyczna

Tablica vs List<T> – porównanie

Cecha	Tablica (int[])	List<int>
Rozmiar	Stały (ustalony przy tworzeniu)	Dynamiczny (rośnie i maleje)
Dodawanie	Brak – ręcznie przepisuj	Add(), Insert(), AddRange()
Usuwanie	Brak – ręcznie przepisuj	Remove(), RemoveAt(), Clear()
Indeksowanie	tab[i]	lista[i] – identycznie
Długość	.Length	.Count
Wydajność	Szybsza (stały bufor)	Nieco wolniejsza
Użycie	Dane o stałej liczbie	Dane o zmiennej liczbie

Deklaracja i inicjalizacja

```
using System.Collections.Generic;

// 1. Pusta lista liczb całkowitych
List<int> liczby = new List<int>();

// 2. Krócej z var (kompilator wywnioskuje typ)
var imiona = new List<string>();

// 3. Inicjalizacja z wartościami od razu
var oceny = new List<int> { 5, 4, 3, 5, 2 };

// 4. Lista własnych obiektów
var uczniowie = new List<Uczen>();
```

Dostęp do elementów i iterowanie

Dostęp przez indeks

```
var owoce = new List<string>
{
    "jabłko", "gruszka",
    "śliwka" };

// Indeksowanie jak tablica
Console.WriteLine(owoce[0]); // jabłko
Console.WriteLine(owoce[2]); // śliwka

// ⚠️ Count, NIE Length!
Console.WriteLine(owoce.Count); // 3
```

Iterowanie po liście

```
// foreach - najczęściej
foreach (var owoc in owoce)

    Console.WriteLine(owoc);

// for - gdy potrzebny indeks
for (int i = 0; i < owoce.Count; i++)
    Console.WriteLine(
        $"{i}: {owoce[i]}");

// Wynik:
// 0: jabłko
// 1: gruszka
// 2: śliwka
```

Dodawanie elementów

```
var lista = new List<int>();

lista.Add(10);           // lista: [10]
lista.Add(20);           // lista: [10, 20]
lista.Insert(0, 5);      // lista: [5, 10, 20] - wstawia na pozycję 0

// AddRange - dodaj wiele naraz
lista.AddRange(new[] { 30, 40 }); // lista: [5, 10, 20, 30, 40]

Console.WriteLine(string.Join(", ", lista)); // 5, 10, 20, 30, 40
```

Usuwanie, sortowanie, odwracanie

Usuwanie

```
var lista = new List<int>
{
    5, 10, 20 };

// Usuwa pierwszą wartość 10
lista.Remove(10);
// lista: [5, 20]

// Usuwa element o indeksie 0
lista.RemoveAt(0);
// lista: [20]

// Usuwa wszystko
lista.Clear();
// lista: []
```

Sortowanie i odwracanie

```
var n = new List<int>
{
    5, 2, 8, 1 };

n.Sort();
// [1, 2, 5, 8] - rosnąco

n.Reverse();
// [8, 5, 2, 1] - malejąco

// Sortowanie niestandardowe
var slowa = new List<string>
{
    "banan", "kot", "arbuz" };

// Lambda: sortuj wg długości
slowa.Sort((a, b) =>
    a.Length.CompareTo(b.Length));
// [kot, banan, arbuz]
```


Przydatne metody – szybki przegląd

```
var lista = new List<int> { 3, 7, 1, 7, 9 };

lista.Contains(7);           // true  - czy zawiera 7?
lista.Contains(99);          // false - 99 nie istnieje

lista.IndexOf(7);            // 1    - indeks PIERWSZEGO wystąpienia
lista.LastIndexOf(7);        // 3    - indeks OSTATNIEGO wystąpienia

lista.Count;                 // 5    - liczba elementów

// Konwersja do tablicy
int[] tab = lista.ToArray(); // int[] { 3,7,1,7,9 }
```

Wyszukiwanie: Find i FindAll

```
var liczby = new List<int> { 3, 15, 7, 22, 11 };
```

```
// Find - zwraca PIERWSZY pasujący element  
int pierwszyDuzy = liczby.Find(x => x > 10);  
// pierwszyDuzy = 15
```

```
// FindAll - zwraca LISTĘ pasujących  
List<int> duze = liczby.FindAll(x => x > 10);  
// duze = [15, 22, 11]
```

```
// Jak to działa? x => x > 10 to wyrażenie lambda  
// x - to każdy element, sprawdzamy warunek dla niego  
// Jeśli warunek spełniony: element trafia do wyniku
```

Smak LINQ – Where, Any, Average

```
using System.Linq;

var oceny = new List<int> { 5, 3, 4, 2, 5, 1 };

// Where - filtruj (zwraca IEnumerable)
var zdane = oceny.Where(o => o >= 2).ToList();
// zdane = [5, 3, 4, 2, 5]

// Any / All
bool jestPiatka    = oceny.Any(o => o == 5);    // true
bool wszystkieOK   = oceny.All(o => o >= 2);    // false

// Agregacje
double srednia = oceny.Average(); // 3.33
int max  = oceny.Max();           // 5
int suma = oceny.Sum();           // 20
```